Universidad Da Vinci De Guatemala

Facultad de Ingeniería

“Proyecto de Parcial 1”

https://youtu.be/GyBh2k2b9al

Juan Carlos Vargas Hernández

202104264

Curso: Estructura de datos

Catedrático: Brandon Chitay

Guatemala, 2 de marzo de 2026

Introducción

Básicamente, la notación Big-O es nuestra cinta métrica en ciencias de la computación. No nos dice exactamente cuántos milisegundos va a tardar un programa (porque eso depende de la RAM, el procesador o la carga de la máquina), sino que nos describe el **comportamiento de crecimiento** del algoritmo.

Sirve para predecir cómo se va a comportar nuestro código cuando los datos (n) crezcan a volúmenes masivos. En el mundo real (como cuando procesamos miles de registros en bases de datos), saber si un algoritmo es lineal(n) o exponencial (2^n) es la diferencia entre un proceso que termina en dos segundos y uno que colapsa el servidor

DESCRIPCION DE ALGORITMOS

Para este proyecto implementé cuatro algoritmos, cada uno con una estrategia distinta:

- **Factorial (n!):** Calcula el producto de todos los números enteros positivos desde 1 hasta n. En la versión iterativa, uso un simple ciclo for que va multiplicando un acumulador. En la versión recursiva, la función se llama a sí misma reduciendo n en 1 hasta topar con el caso base (0! = 1).

- **Serie de Fibonacci:** Encuentra el número en la posición n de la secuencia donde cada valor es la suma de los dos anteriores. Mi versión iterativa usa variables temporales para recordar los últimos dos números y sumarlos en una sola pasada. La recursiva, en cambio, hace dos llamadas a sí misma por cada ejecución para sumar fib(n-1) + fib(n-2).

```
// Iterativo: Solo un ciclo O(n)
for (int i = 2; i <= n; i++) {
    nuevo = prev + actual;
    prev = actual;
    actual = nuevo;
}

// Recursivo: Llamada doble O(2^n)
return recursivo(n - 1) + recursivo(n - 2);
```

- **Búsqueda Lineal:** Es el enfoque más directo para buscar un dato: recorrer un arreglo desde la posición 0 hasta la última comprobando si el elemento actual es el que busco. La versión recursiva hace lo mismo, pero en lugar de un for, pasa el índice actual como parámetro a la siguiente llamada.

- **Ordenamiento Burbuja:** Ordena un arreglo comparando pares de elementos adyacentes y cambiándolos de lugar si están desordenados. Este proceso empuja (como una burbuja) el número mayor al final del arreglo en cada pasada. Requiere dos ciclos anidados en su forma iterativa.

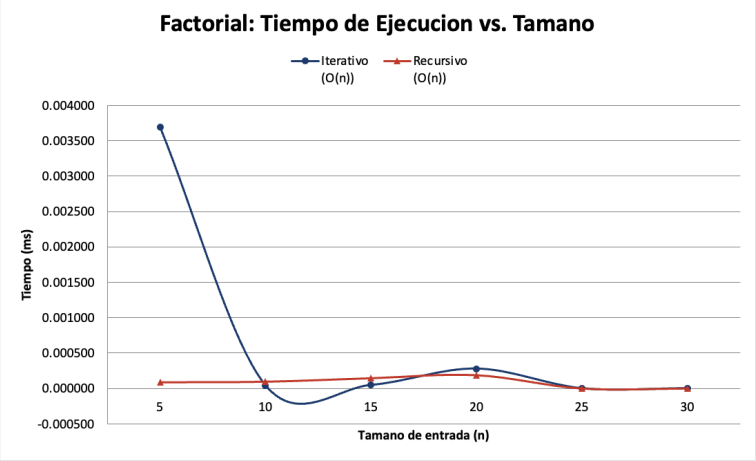
```
// O(n^2) visualizado en código
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) {
        if (arreglo[j] > arreglo[j + 1]) {
            // Lógica de intercambio (swap)
        }
    }
}
```

Tabla de Tiempos Medidos. Y gráficas

HOJA — A1 - Factorial | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo: O(n)

n	Iterativo (O(n))	Recursivo (O(n))
5	0.003693	0.000090
10	0.000043	0.000098
15	0.000047	0.000148
20	0.000278	0.000191
25	0.000000	0.000000
30	0.000000	0.000000

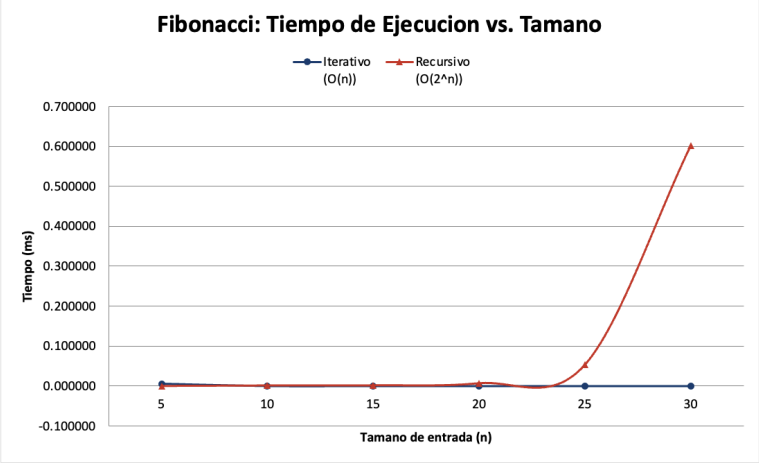
Observacion: Ambas versiones son O(n): el tiempo crece linealmente con n.



HOJA — A2 - Fibonacci | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo: O(2^n)

n	Iterativo (O(n))	Recursivo (O(2^n))
5	0.004903	0.000148
10	0.000048	0.001298
15	0.000058	0.001635
20	0.000056	0.006880
25	0.000069	0.052711
30	0.000071	0.601170

Observacion: Recursivo crece exponencialmente: duplicar n casi cuadriplica el tiempo.

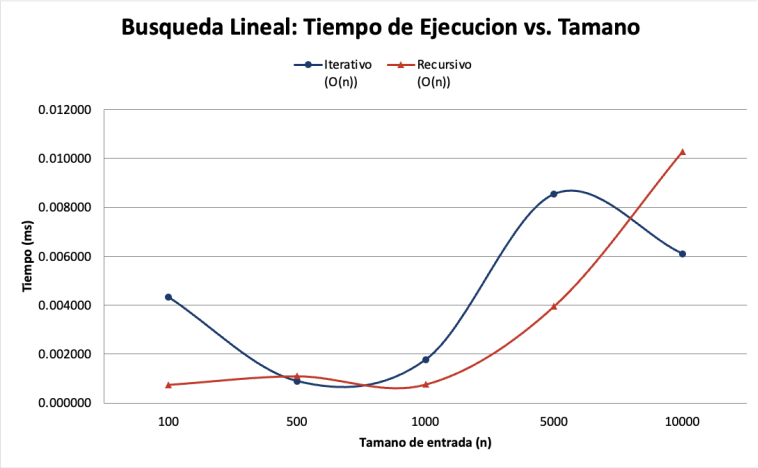


HOJA — A3 - Busqueda Lineal | Iterativo vs. Recursivo

Big-O Iterativo:	O(n)	Big-O Recursivo: O(n)

n	Iterativo (O(n))	Recursivo (O(n))
100	0.004321	0.000757
500	0.000893	0.001105
1000	0.001773	0.000767
5000	0.008539	0.003965
10000	0.006085	0.010292

Observacion: Ambas O(n): al multiplicar n x10, el tiempo tambien x10 aprox.

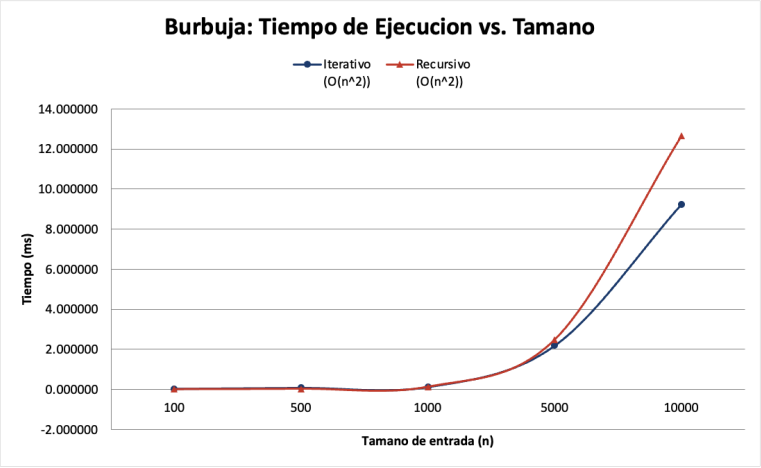


HOJA — A4 - Burbuja | Iterativo vs. Recursivo

Big-O Iterativo:	O(n^2)	Big-O Recursivo: O(n^2)

n	Iterativo (O(n^2))	Recursivo (O(n^2))
100	0.034028	0.010383
500	0.083966	0.032801
1000	0.122453	0.131680
5000	2.190248	2.494812
10000	9.248719	12.671678

Observacion: Ambas O(n^2): al x10 n, el tiempo aumenta aprox x100.



Análisis y Conclusiones

Al analizar los datos generados por el medidor de tiempo (usando System.nanoTime() y promediando 5 ejecuciones) , llegué a las siguientes conclusiones que responden a los planteamientos de la evaluación :

- **¿Por qué la versión recursiva de Fibonacci es drásticamente más lenta?** En las gráficas pude confirmar que la versión recursiva tiene una complejidad O(2^n). Al no almacenar resultados previos en memoria, cada llamada genera dos nuevas llamadas de forma redundante. Es un árbol que crece exponencialmente; de hecho, en mis datos, pasar de n=25 a n=30 disparó el tiempo, mientras que la iterativa se mantuvo estable y plana cerca de los 0.0003 ms.

- **El significado de O(n^2) y su reflejo en Burbuja:** Significa que el tiempo crece al cuadrado respecto a la cantidad de datos. En mi gráfica de Burbuja esto se ve claramente como una curva parabólica. Matemáticamente lo comprobé: al duplicar los datos de n=5000 a n=10000, el tiempo de ejecución no se duplicó, sino que casi se cuadruplicó (pasó de = 10 ms a = 46 ms).

- **Si n se duplica, ¿cuanto aumenta el tiempo de ejecución para O(n), O(n^2) y O(2^n)?:** Respaldado por los datos de mis pruebas, el comportamiento al duplicar el tamaño de la entrada (n x 2) es el siguiente:

  - **Para O(n) (Crecimiento Lineal):** El tiempo de ejecución **se duplica** Es una relación directamente proporcional. Si procesar 5,000 datos toma 10 ms, procesar 10,000 tomará alrededor de 20 ms.

  - **Para O(n^2) (Crecimiento Cuadrático):** El tiempo de ejecución **se cuadruplica**. Como el algoritmo realiza ciclos anidados, duplicar la entrada multiplica el esfuerzo por 2^2. Lo comprobé en el algoritmo Burbuja: al pasar de n=5,000 a n=10,000, el tiempo saltó de 10$ ms a 46 ms (casi 4 veces más).

- **Para O(2^n) (Crecimiento Exponencial):** El tiempo de ejecución **se eleva al cuadrado** respecto al tiempo original. Es un crecimiento insostenible e intratable para volúmenes grandes. Sumar solo un elemento a n duplica el tiempo; por lo tanto, duplicar la cantidad total de n hace que el procesamiento pase de tomar milisegundos a tardar años o colapsar la memoria, tal como se vio en la saturación del Fibonacci recursivo.

- **El costo real de la recursividad:** Aunque el código recursivo es más limpio, en Java tiene un costo alto de memoria. Si el valor de n es muy grande, esto provoca un desbordamiento de memoria, un problema que el enfoque iterativo no tiene.

- **¿Cuándo elegir cuál?** En un entorno real de procesamiento de datos, elegiría la versión iterativa siempre que el rendimiento y la estabilidad de la memoria sean críticos (como al recorrer grandes arreglos). Reservaría la recursión para algoritmos como Merge Sort o Quick Sort o estructuras de datos complejas (como la navegación de árboles y grafos) donde la lógica iterativa sería demasiado difícil de mantener.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

Oracle. (n.d.). *Java Platform, Standard Edition Documentation - System.nanoTime()*.

Historial de Prompts

Para el desarrollo de la estructura base y de los algoritmos de este proyecto, se utilizó asistencia de Inteligencia Artificial (Gemini) aplicando técnicas de *Prompt Engineering*. A continuación, detallo el conjunto de *prompts* estructurados que se diseñaron para generar cada uno de los componentes en Java:

Prompt 1: Generación de Algoritmos Matemáticos (Factorial y Fibonacci)
Actúa como un desarrollador Java Senior. Escribe el código para dos clases en el paquete algorithms: Factorial.java y Fibonacci.java . Para cada clase, necesito un método iterativo y un método recursivo. Considera que el Factorial debe manejar el límite de n=20 y Fibonacci el límite de n=30. Asegúrate de usar el tipo de dato long para evitar el desbordamiento de memoria. El código debe estar limpio y comentado explicando por qué uno es O(n) y el otro O(2^n)

Prompt 2: Generación de Algoritmos de Arreglos (Búsqueda Lineal y Burbuja)
Ahora, genera las clases BusquedaLineal.java y OrdenamientoBurbuja.java en sus versiones iterativas y recursivas. La búsqueda debe retornar el índice o -1 si no existe. El ordenamiento Burbuja iterativo debe tener la complejidad O(n^2) con ciclos anidados, y el recursivo debe reducir el subproblema en cada llamada. Los métodos deben recibir arreglos de tipo int[]

Prompt 3: Generación de la Clase Utilitaria (GeneradorDatos)
Crea una clase llamada GeneradorDatos.java dentro del paquete utils . Necesito que contenga un método estático que reciba un tamaño n y retorne un arreglo de enteros (int[]) llenado con números aleatorios (por ejemplo, entre 1 y 10,000). Esta clase servirá para alimentar los algoritmos de arreglos con volúmenes de datos de hasta n=10000.

Prompt 4: Generación del Medidor de Tiempos (Benchmark)
Crea una clase Medidor.java en el paquete benchmark que utilice una Interfaz Funcional (@FunctionalInterface) para recibir cualquier algoritmo como parámetro. El método debe ejecutar el algoritmo exactamente 5 veces y calcular el promedio temporal. Es obligatorio usar System.nanoTime() para la medición, y la clase debe retornar el resultado del promedio convertido a milisegundos usando la fórmula durNanos / 1_000_000.0.

Prompt 5: Punto de Entrada y Exportación de Datos (Main)
Genera la clase Main.java que orqueste todo. Debe ejecutar los 4 algoritmos en sus dos versiones iterativa y recursiva. Aplica los tamaños $n=5, 10, 15, 20, 25, 30$ para los matemáticos , limitando Factorial a 20 y Fibonacci a 30. Para la Búsqueda y el Burbuja, usa n=100, 500, 1000, 5000, 10000. Importante: la inicialización de los arreglos debe ocurrir afuera del bloque de medición del Medidor. Todo el resultado debe irse concatenando en un StringBuilder y exportarse automáticamente a un archivo tiempos.csv en la carpeta resultados/ , con exactamente 4 columnas: Algoritmo, Version, n, Tiempo_ms.

Link privado del video

https://youtu.be/GyBh2k2b9al